Solution: Root Equals Sum of Children

Problem Statement

Given a binary tree with exactly 3 nodes (single root and two child nodes), determine if the `root's value` is equal to the `sum of the child nodes` values.

Examples

- **Example 1**:
  - **Input**: root = [10, 3, 7]
  - **Expected Output**: true
  - **Justification**: The root's value (10) is equal to the sum of its children's values (3 + 7).
- **Example 2**:
  - **Input**: root = [4, 1, 5]
  - **Expected Output**: false
  - **Justification**: The root's value (4) is not equal to the sum of its children's values (1 + 5).
- **Example 3**:
  - **Input**: root = [15, 10, 5]
  - **Expected Output**: true
  - **Justification**: The root's value (15) is equal to the sum of its children's values (10 + 5).

Solution

To solve this problem, we'll perform a straightforward comparison between the root node's value and the sum of its children's values. This approach is effective because the problem's scope is limited to a root with only two children, simplifying the verification process. By directly accessing the root and its children's values, we can determine the truth of the statement with minimal computation, making this method both efficient and straightforward.

This approach ensures accuracy and performance by eliminating the need for traversing or searching through additional nodes, as the problem guarantees the presence of exactly two children for the root node.

Step-by-step Algorithm

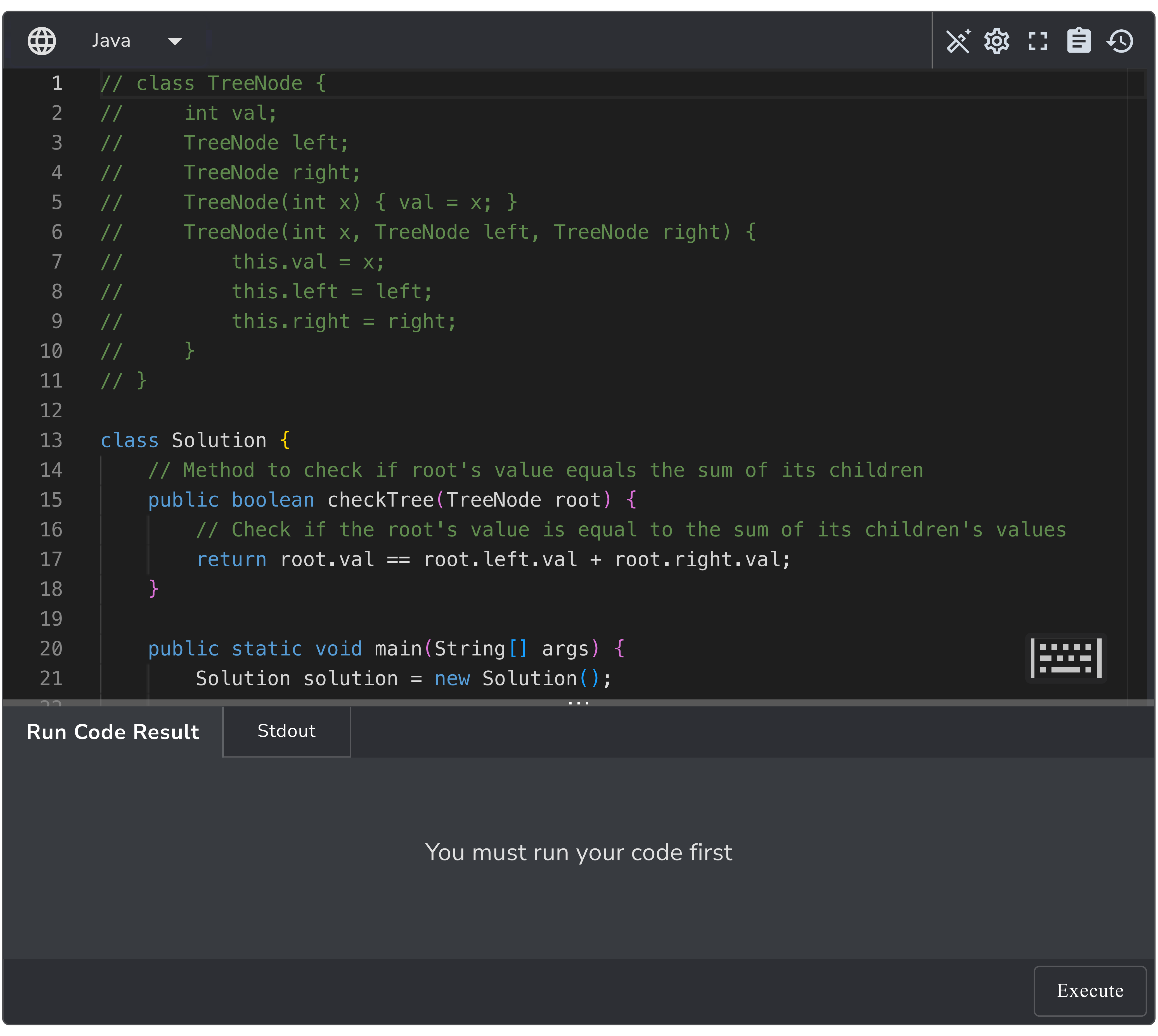
- **Step 1**: Access the value of the root node.
- **Step 2**: Access the values of the root's two children nodes.
- **Step 3**: Sum the values of the two children nodes.
- **Step 4**: Compare the sum of the children's values with the root's value.
- **Step 5**: Return `true` if the root's value equals the sum of its children's values; otherwise, return `false`.

Algorithm Walkthrough

Given the input `root = [10, 3, 7]`:

- **Step 1**: The root's value is 10.
- **Step 2**: The first child's value is 3, and the second child's value is 7.
- **Step 3**: The sum of the children's values is 3 + 7 = 10.
- **Step 4**: Compare the root's value (10) with the sum of its children's values (10).
- **Step 5**: Since the root's value equals the sum of its children's values, return `true`.

Code



```java
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
//     TreeNode(int x, TreeNode left, TreeNode right) {
//         this.val = x;
//         this.left = left;
//         this.right = right;
//     }
// }

class Solution {
    // Method to check if root's value equals the sum of its children
    public boolean checkTree(TreeNode root) {
        // Check if the root's value is equal to the sum of its children's values
        return root.val == root.left.val + root.right.val;
    }

    public static void main(String[] args) {
        Solution solution = new Solution();
```

You must run your code first

Complexity Analysis

Time Complexity

$O(1)$: The algorithm performs a constant time operation, as it directly accesses the root and its two children without traversing any additional nodes. This means the execution time remains constant regardless of the input size.

Space Complexity

$O(1)$: The space complexity is also constant because the algorithm uses a fixed amount of space. It does not allocate any extra data structures that grow with the input size.

← Previous
Root Equals Sum Of Children (Easy)

Next →
Excel Sheet Column Title (Easy)

☐ Mark as Completed